MeetMyPets

From P(doom) to MeetMyPets: a learning guide

How a music-video engine became a 30-second, interactive app launch film, written in code. How it works, how it was built, how to turn it into an AI video generator, and how to use it to grow as an AI engineer.

For a fresher AI engineer · October 2026 · Chapters 1–9 + glossary

Contents

0 How to use this guide

1 The big picture

1.1 What did we make? · 1.2 The one idea that makes it all work · 1.3 Why build video with code at all? · 1.4 The two lives of this engine · 1.5 The layers, from the outside in · 1.6 What makes it "AI engineering"?

2 How the engine works: from a browser tab to an MP4

2.1 The tools we stand on (the "platform") · 2.2 Boot: what happens when the page opens · 2.3 A scene: the unit of the film · 2.4 Rendering one frame (the heart) · 2.5 Motion blur: why a frame is many frames · 2.6 How our shots draw (the Shot base class) · 2.7 Export: how frames become an MP4 · 2.8 The 2D/3D toolbox, in one table · 2.9 Check yourself

3 From P(doom) to MeetMyPets: the journey

3.1 Where we started: the P(doom) engine · 3.2 Stage 1: the plan (before any code) · 3.3 Stage 2: brand and references · 3.4 Stage 3: real screens instead of mock-ups · 3.5 Stage 4: the first build: an "animatic" · 3.6 Stage 5: the interactive-widget idea · 3.7 Stage 6: the 30-second cut · 3.8 Stage 7: the enhancement pass ("lots of enhancement") · 3.9 Stage 8: making rendering work on your PC · 3.10 The final pipeline

4 Music and timing

4.1 Music words, simply · 4.2 Sound as numbers · 4.3 How we measured the beat grid (no AI needed) · 4.4 How P(doom) did it: the AI audio pipeline · 4.5 From measurements to the film · 4.6 Exercises

5 The shots, one by one, and the building blocks

5.1 Three coordinate systems (read this first) · 5.2 The building blocks ("kits") · 5.3 The nine shots · 5.4 Tutorial: change something yourself

6 Running, rendering, debugging

6.1 One-time setup (already done on your PC) · 6.2 Everyday commands (run inside `app/`) · 6.3 The full render, step by step · 6.4 War stories: every problem we hit and how we solved it · 6.5 A debugging checklist

7 Turning this into a video generator

7.1 The vision · 7.2 What we already have (more than you think) · 7.3 What's missing · 7.4 Target architecture · 7.5 The AI parts in detail · 7.6 Phased roadmap · 7.7 Build vs reuse · 7.8 Risks to plan for · 7.9 Your first three tasks, if you start tomorrow

8 Your career guide: explaining this project and growing from it

8.1 What this project actually proves about you · 8.2 The pitches · 8.3 STAR stories (Situation, Task, Action, Result) · 8.4 Interview questions with answers · 8.5 Be honest about what you did · 8.6 Gaps to fill (honest skills map) · 8.7 A 12-month plan for a fresher AI engineer · 8.8 Portfolio and job-search tips

9 Glossary

START HERE

How to use this guide

This guide is written for **you**, a fresher AI engineer. After reading it you should be able to:

1. **Understand** every part of this project: how a video can be *programmed* instead of edited.
2. **Explain** it confidently: in a 30-second pitch, a 2-minute walkthrough, or a deep technical interview.
3. **Change** it yourself: add a shot, change timing, fix a render problem.
4. **Grow** it into a product: an AI-powered *video generator*.
5. **Grow your career** from it: what you learned, what to learn next, and how to show it.

Reading order

#	Chapter	What you get	Time
1	The big picture	What this project is, in plain words; the one idea that makes it work	20 min
2	How the engine works	From a browser tab to an MP4 file, step by step	60 min
3	From P(doom) to MeetMyPets	Everything that changed, why, and in which file: the journey	45 min
4	Music and timing	Beats, bars, the drop; how we *measured* the music, and how P(doom) used AI to do it	40 min
5	The shots, one by one	Each of the 9 shots and the building blocks (widgets, worlds, paw taps); a tutorial to add your own	60 min
6	Running, rendering, debugging	Every command, and every problem we hit with its fix (real war stories)	30 min
7	Turning this into a video generator	Architecture, the AI parts, phases from MVP to product	45 min
8	Your career guide	How to present this project, interview Q&A, skills map, a 12-month plan	45 min
9	Glossary	Every jargon word, in one line	reference

Tip: read chapters 1 → 2 → 3 first. Keep the code open next to the guide; every chapter links to the real files.

The project in one breath

> We built a **30-second, 60 fps launch film for the MeetMyPets app entirely in code**. The film is a web page that can draw *any moment in time* on demand. A script opens that page in a hidden browser, asks it for frame 0, 1, 2 … 1834, and pipes the pictures into **ffmpeg**, which makes the MP4. Everything — camera moves, 3D cards made from the real Figma screens, paw taps, typing, sound effects — is a **function of time**, locked to beats we **measured** from the music.

Where things are

```
meet-my-pets/
├── audio/testing.mp3          the music (the film uses 45.55 s → end of it)
├── data/audio.json            the beat grid the engine reads (written by prep-launch.ts)
├── docs/launch-film/          the production plan: storyboard, prompts, analysis
├── docs/learn/                ← this guide
├── analysis/                  P(doom)'s Python tools: AI music + lyric analysis
├── out/                       rendered videos (MeetMyPets_30s_v3.mp4 = the film)
└── app/                       the video engine (a web app)
    ├── index.html             the page
    ├── scripts/
    │   ├── prep-launch.ts     music cut + beat grid
    │   ├── render.ts          headless browser → frames → ffmpeg → MP4
    │   └── sfx.ts             sound effects synthesised in code, mixed under the music
    └── src/
        ├── main.ts            boots the engine (preview player or export mode)
        ├── timeline.ts        reads the storyboard → which scene draws which seconds
        ├── engine/            the core: rendering, post effects, audio data, fonts, maths
        └── scenes/            the film: film-s01.ts … film-s09.ts + shared kits (_shot, _ui, _kit, _world,
_mark)
```

CHAPTER 1

1. The big picture

1.1 What did we make?

A **30-second launch film** for MeetMyPets, a social app for pets (Discover pets, like, match, chat). The film tells a story:

> Bubbly (your golden retriever) signs up → gets a profile → sees Lily's post in the Feed → finds Lily in Discover → they **match** on the music's drop → they chat → logo.

Every app feature is shown **interactively**: a piece of the real app (cut from the Figma designs) lifts out into 3D space, a paw taps it, and it reacts exactly as the app would.

The surprising part: **nobody opened a video editor**. There is no After Effects or Premiere project. The whole film is **TypeScript code** running in a web browser.

1.2 The one idea that makes it all work

> **A video is just a function of time.**
>
> `picture = draw(t)` where `t` is the time in seconds.

If you can write a function that, for *any* time `t`, draws exactly what the film shows at that moment, then:

- **Preview** = call `draw(t)` 60 times a second in the browser while the music plays.
- **Export** = call `draw(0)`, `draw(1/60)`, `draw(2/60)` … `draw(30.583)`, save each picture, glue them into an MP4.
- **Seek** = call `draw(17.3)` directly. No need to "play up to" 17.3 s.

This is called **deterministic rendering**: the same `t` always gives the same picture. That's why every animation in the code is written like this:

```
// the card arrives over 0.5 s starting at 0.2 s, with a smooth "ease out"
const k = ease.outExpo(prog(lt, 0.2, 0.7));     // k goes 0 → 1
card.position.x = lerp(2.6, 0.5, k);            // from x = 2.6 to x = 0.5
```

`prog(t, a, b)` turns "where are we between a and b" into a number from 0 to 1. `ease.outExpo` makes the motion fast at first and gentle at the end (like a real object). `lerp(a, b, k)` blends between two values. **These three tiny helpers drive almost every motion in the film.** (They live in `app/src/engine/util.ts`.)

> **Simple analogy:** a flip-book. Each page is drawn from a rule ("on page 47 the dog is here"), not by copying the page before. You can open the book at any page.

1.3 Why build video with code at all?

Editor (After Effects, Premiere)	Code (this project)
Every frame placed by hand	Every frame computed from rules
Change the music → re-time everything by hand	Change the beat grid → everything re-times itself
Hard to make 100 variants	Change a JSON file → a new video (→ a **video generator**)
Big binary project files	Text files: Git, code review, diffs
Great for one-offs	Great for **templates and automation**

This is the same idea behind tools like **Remotion** (React video) and **Motion Canvas**. Our engine is a custom one, built first for a music video.

1.4 The two lives of this engine

Life 1: the P(doom) music video. The engine was first built for a music video called *"I'm Upping My P(doom)"* (a song about AI risk). That video is 2.5 minutes of "illustrated plates": an eye, a spark that writes lyrics, paperclips, a fuse … all drawn in code and synced **word by word** to the singing. Its style bible is in `docs/TREATMENT.md` and its engine manual in `docs/ENGINE.md`. To sync to the singing, it used **AI audio tools** (Demucs, Whisper, CTC alignment; see chapter 4).

Life 2: the MeetMyPets launch film (this project). We took the same engine (the repo's first commit is literally *"Start from the pdoom-video engine"*) and turned it into a **product launch film**:

```
P(doom) music video                       MeetMyPets launch film
──────────────────                        ──────────────────────
song with lyrics, 2.5 min          ──►    instrumental track, 30 s cut
synced to WORDS (AI alignment)     ──►    synced to BEATS (measured grid)
abstract illustrated plates        ──►    the real app UI as 3D "widgets"
1920×1080                          ──►    1920×1080 (briefly 9:16, then back for YouTube)
dark orange "doom" palette         ──►    coral / orange / amber brand palette
~30 hand-written scenes            ──►    9 scenes on shared "kits" + a storyboard JSON
```

Chapter 3 tells this journey step by step.

1.5 The layers, from the outside in

```
┌──────────────────────────────────────────────────────────────────┐
│ 5. STORY      storyboard.launch.json — shots, times, copy        │  (what happens when)
├──────────────────────────────────────────────────────────────────┤
│ 4. SHOTS      scenes/film-s01.ts … film-s09.ts                   │  (each shot's choreography)
├──────────────────────────────────────────────────────────────────┤
│ 3. KITS       _shot (base) _ui (widgets) _kit (stickers,         │  (reusable building blocks)
│               keyboard, banner) _world (colour + decor) _mark    │
├──────────────────────────────────────────────────────────────────┤
│ 2. ENGINE     engine.ts timeline + motion blur + post effects    │  (from P(doom))
│               gl.ts layers · type.ts fonts · audio.ts beats      │
├──────────────────────────────────────────────────────────────────┤
│ 1. PLATFORM   browser · WebGL · three.js · Vite · Bun · ffmpeg   │  (tools we stand on)
└──────────────────────────────────────────────────────────────────┘
```

When you explain the project, go **top-down** (story → shots → kits → engine → platform). When you debug, go **bottom-up**: is the platform OK (GPU, ffmpeg)? Then the engine? Then my shot?

1.6 What makes it "AI engineering"?

People sometimes think AI engineering = calling a chatbot API. Real AI engineering is building **systems** where AI and normal code work together. This project touches several parts of that:

1. **AI as a collaborator**: the whole build was done with an AI coding agent (Claude Code). You directed it, reviewed its renders, and corrected its course ("too slow", "like a slideshow", "build uniquely"). Directing an agent well is a core modern skill.
2. **AI in the pipeline**: P(doom) used AI models (Demucs for source separation, Whisper and CTC models for word alignment) to turn raw audio into timing data.
3. **Signal processing without AI**: for MeetMyPets we *measured* the beat grid with plain maths (loudness per frequency band), because a simple, explainable method was enough. Knowing **when not to use a model** is an engineering skill too.
4. **The next step**: a video *generator*, where a language model writes the storyboard JSON and the engine renders it (chapter 7).

CHAPTER 2

2. How the engine works: from a browser tab to an MP4

This chapter follows **one frame** from code to file. Read it slowly, with the files open.

2.1 The tools we stand on (the "platform")

Tool	What it is	Why we use it
TypeScript	JavaScript with types (`x: number`)	Catches mistakes before running; easier to read
Browser + WebGL	Chrome's built-in way to use the GPU (graphics card) from JavaScript	Fast drawing of millions of pixels, 3D, shaders
three.js	A library on top of WebGL: scenes, cameras, meshes, lights, materials	Writing raw WebGL is like assembly; three.js is like a framework
Canvas 2D	The browser's simple drawing API (`fillText` , `arc` , `drawImage`)	Text, UI drawings, paw taps. Easy and crisp
Vite	A dev server + bundler	Serves the app at `localhost` , reloads on save
Bun	A fast JavaScript runtime (like Node.js)	Runs our scripts: `prep-launch.ts` , `render.ts` , `sfx.ts`
Playwright	Controls a real browser from a script	Opens the film page **headless** (invisible) for rendering
ffmpeg	The Swiss-army knife of audio and video	Turns raw frames into H.264 MP4, cuts and mixes audio

> **GPU vs CPU, simply:** the CPU is a few very smart workers; the GPU is thousands of simple workers. Colouring 2 million pixels, 60 times a second, is a perfect job for thousands of simple workers.

2.2 Boot: what happens when the page opens

File: `app/src/main.ts` → `app/src/engine/engine.ts` `init()`

1. **Load the data**: `data/audio.json` (the beat grid, length), `data/lyrics.json` (empty for us; P(doom) had every sung word), and all **fonts**.
2. **Build the timeline**: `timeline.ts` reads `storyboard.launch.json` and returns a list of **entries**: `{ id: 'S04', start: 9.933, end: 13.244, load: () => import('./scenes/film-s04.ts') }` .
3. **Load each scene**: for every entry, import its file, create the scene object, and run its `init()` (load images, cut widgets, build 3D objects). This happens **once**, not every frame.
4. Then either start the **preview player** (play/pause, scrub bar) or wait for the **export script** to ask for frames (`?export=1` in the URL).

2.3 A scene: the unit of the film

Every shot is a **Scene** (`engine/scene.ts`): a class with two important methods:

```
class MyShot extends Scene {
  async init() { /* once: load images, build objects */ }
  render(f: Frame, out: RenderTarget) { /* every frame: draw time f.t into `out` */ }
}
```

`f` (the **Frame**) carries everything a scene needs to know about "now":

Field	Meaning
`f.t`	film time in seconds
`f.lt`	**local** time: seconds since *this shot* started (most of our code uses this)
`f.p`	progress through the shot, 0 → 1
`f.beat` , `f.beatPhase`	which beat we're on, and how far into it (0 → 1). Used for the beat punch
`f.a`	audio features (loudness, bass ...)

`out` is a **render target**: an invisible picture in GPU memory that the scene must fill completely.

2.4 Rendering one frame (the heart)

`engine.render(t)` → `composite(t)` :

```
   time t
     |
     ▼
which timeline entries cover t?          ← usually one; two during an overlap
     |
     ▼
scene.render(frame, rt)                   ← each draws into its own HDR render target
     |      (our shots: backdrop → 3D stage → 2D overlay; see 2.6)
     ▼
transition mix (if two scenes overlap)    ← fade / zoom / whip / flash shader
     |
     ▼
motion blur accumulation (export only)    ← render several sub-frames, average them (2.5)
     |
     ▼
post-processing  (post.ts)                ← exposure, bloom (glow), vignette, grain,
     |                                      chromatic aberration, ZOOM (our beat punch)
     ▼
HUD layer (hud.ts)                        ← P(doom)'s readout; switched off in our film
     |
     ▼
final image on the canvas (sRGB)
```

HDR render targets. The scenes draw into *HalfFloat* targets: colours can go above 1.0 ("brighter than white"). Post-processing uses that for **bloom**: anything brighter than a threshold glows. It also explains why we turned bloom down for the white app screens: they would have glowed.

Linear vs sRGB colour. Inside the engine colours are **linear** (maths works correctly: 0.5 really is half the light). At the very end they're converted to **sRGB**, the curve screens expect. Canvas 2D layers are sRGB and

get converted when uploaded. *If colours ever look washed out or too dark, suspect a colour-space mix-up.*

2.5 Motion blur: why a frame is many frames

A real camera keeps its shutter open for a moment, so fast things smear. Our engine imitates that. To make **one** frame it renders **several sub-frames** at slightly different times inside the "shutter" and **averages** them:

```
frame at t = 10.000 s, shutter 0.3 → sub-frames spread over ±0.0025 s
  t=9.9975 ─┐
  t=9.9992 ─┤  average  →  one frame with a smooth motion streak
  t=10.0008 ┤
  t=10.0025 ┘
```

- `--samples 2` → 2 sub-frames. Fast objects show **two copies** (we saw this as "doubled text").
- `--samples 4` → 4 copies, closer together.
- `--samples auto` → **adaptive**: the engine keeps adding sub-frames (4, 12, 36 …) only where the picture still changes. It divides the frame into **32×32 tiles** and stops refining tiles that have converged. Converged tiles are skipped cheaply through the GPU depth buffer ("early-Z"). Our final film used `auto` up to 36, so every fast move is a smooth streak.

This is why the engine insists on **determinism**: sub-frames are rendered *out of order*, so a scene must never depend on "what I drew last time". It must depend only on `t`.

2.6 How our shots draw (the Shot base class)

All MeetMyPets shots extend `scenes/_shot.ts`, which fixes the order of layers:

```
1. backdrop   (Canvas 2D, drawn once)        the act's colour gradient   (_world.ts)
2. 3D stage   (three.js scene)               widgets, phone, stickers, floating decor, floor shadows
       └─ rendered into a 4× MSAA target (smooth edges), then laid over the backdrop
3. overlay    (Canvas 2D, every frame)       headline text, paw taps, flashes, wipes
4. post       (returned settings)            bloom, vignette, grain, and `zoom` = the beat punch
```

A subclass only writes three methods:

- `setup()`: load images, cut widgets, build the world
- `frame(lt)`: move the camera and widgets; repaint widget states
- `overlay(c, lt)`: draw 2D on top

2.7 Export: how frames become an MP4

File: `app/scripts/render.ts`



Words you'll meet:

- **Headless browser**: a real Chrome with no window. It still uses the GPU.
- **WebSocket**: a two-way pipe between the page and the script, used to stream big frames fast.
- **Reorder buffer**: two workers finish chunks out of order; frames are held until the next one in sequence arrives.
- **vflip**: WebGL reads pixels bottom-up, so ffmpeg flips them.
- **BT.709**: the colour standard for HD video. Tagging it correctly keeps YouTube's colours identical to the preview.
- **CRF 17**: H.264's quality knob (lower = better and bigger; 17–18 is "visually lossless").

2.8 The 2D/3D toolbox, in one table

File	What it gives you
`engine/gl.ts`	`Layer2D` (a Canvas 2D surface → GPU texture), `makeRT` (render targets), `Compositor.draw` (lay one image over another), `W`, `H` (1920×1080)
`engine/post.ts`	bloom, vignette, grain, chromatic aberration, zoom, shake, flash, fade
`engine/type.ts`	fonts (Archivo, Plex Mono, Cormorant + our Bricolage, Nunito Sans, Love Ya Like A Sister), text measuring
`engine/util.ts`	`clamp`, `lerp`, `prog`, `ease.*`, seeded random (`mulberry32`)
`engine/audio.ts`	beats/bars from `data/audio.json`, `beatAt(t)`
`scenes/_cine.ts`	`Phone3D` (metal phone with a live screen), `makeStage` (lights + camera), backdrops

2.9 Check yourself

1. Why can the preview jump straight to 17.3 s?
2. Why did `--samples 2` show doubled text, and what fixed it?
3. In which order are the backdrop, the 3D stage and the headline drawn, and why that order?
4. What would break if a scene stored "the position from the last frame" in a variable and moved it a bit each frame?

Answers

1. Every frame is a pure function of `t` (deterministic), so no history is needed.
2. 2 sub-frames = 2 visible copies of fast objects. Adaptive sampling (`--samples auto`) adds sub-frames until the streak is continuous.
3. Back to front, like painting: background first, then 3D objects, then text on top so it's never hidden.
4. Sub-frames are rendered out of order and the preview can seek, so that position would be wrong (and differ between preview and export). Compute position from `t` instead.

CHAPTER 3

3. From P(doom) to MeetMyPets: the journey

This is the story of how a music-video engine became a product-launch film, **in the order it happened**, with the reason for each decision. Real projects look like this: plans change, feedback arrives, problems appear. Learning *why* each change happened teaches more than the final code alone.

3.1 Where we started: the P(doom) engine

The engine was built for the music video *"I'm Upping My P(doom)"*. What it already had:

- **Deterministic rendering** of any time `t` (chapter 2), preview + export with the same code.
- **Motion blur** by sub-frame averaging, with adaptive per-tile refinement.
- **Post-processing**: bloom, halation, grain, chromatic aberration, vignette, flash, zoom, shake.
- **A HUD** that showed a "P(doom)" number ticking up through the song.
- **Lyrics data**: every sung word with its start/end time, produced by an **AI audio pipeline** (chapter 4).
- **~30 hand-written scenes** ("plates"): `loss.ts`, `fuse.ts`, `shoggoth.ts`, `room.ts` … each a little art piece.
- **Export** via headless Chrome → WebSocket → ffmpeg, at 1080p or 4K.

All of that is still in the repo: the old scenes sit in `app/src/scenes/` (they're simply not in the timeline any more). That is normal in real projects: you build on a strong foundation instead of starting from zero.

3.2 Stage 1: the plan (before any code)

Request: a professional launch film for MeetMyPets, with a full production package.

What we did first, **before building**:

1. **Measured the music** instead of guessing a BPM (chapter 4). The old `data/audio.json` said 97 BPM and was **wrong**: it had picked up a dotted rhythm. The real grid: a bar every **3.311 s**, the drop at **47.21 s** in the full track.
2. **Read the real app's code** (the Flutter project) to know which features *really exist*, so we wouldn't invent features. Chat, match, feed, discover and create profile are real; health tracking and booking are not, so they're not shown.
3. Wrote the production plan: storyboard, image prompts, workflow, checklist → `docs/launch-film/`.

Lesson: *measure, don't assume; verify features against the source of truth.* That's an engineering habit, not a video habit.

3.3 Stage 2: brand and references

- **The logo**: the double-loop was rebuilt as clean SVGs from the app's own vector file, plus a "woven" film version → `app/public/assets/brand/`.
- **A reference video** (a SaaS launch ad) was studied frame by frame. Your feedback: *"I don't need the exact thing … build uniquely."* So we kept only its **principles**:

- morph instead of cut
- one idea per beat
- the logo earned from something seen earlier

and invented our own devices: the **Two Loops** (Bubbly's coral loop + Lily's amber loop = the logo), **paw-taps**, **collar-tag type**, and **swipe cuts**.

Lesson: use references for *principles*, never for copying.

3.4 Stage 3: real screens instead of mock-ups

- You exported the **Figma screens** → `app/public/assets/figma/`. We analysed every file (`SCREEN_ANALYSIS.md`) and found sample-data problems ("Ruby · Maine Coon" on a dog photo, a leftover "Error message" note).
- The missing **"It's a Match!"** screen was **rebuilt in HTML** from the app's exact strings and rendered to a 4× PNG → `figma_rebuilt/match.html`.
- Fonts: the app's **Bricolage Grotesque** and **Nunito Sans**, plus **Love Ya Like A Sister** for the wordmark (your choice).

3.5 Stage 4: the first build: an "animatic"

Changes:

- `engine/gl.ts`: the frame size (it went to 9:16 and then back to **16:9** when you said *"this is for YouTube"*).
- `scripts/prep-launch.ts`: writes the **measured** beat grid and the soundtrack.
- `timeline.ts`: reads the **storyboard JSON** directly, so one file drives the edit.
- `scenes/film.ts`: an **animatic**, a rough version of all shots: the real screens on a 3D phone plus text.

Your feedback: *"a single mobile coming from down to up is boring ... make the app interactive ... like a story from sign-in."*

Lesson: an animatic is *meant* to be criticised early. It is cheap to change direction at this point.

3.6 Stage 5: the interactive-widget idea

The key invention: `scenes/_ui.ts`, the **widget kit**.

> A **Widget** is a rectangle cut **pixel-exact** from a Figma export (e.g. the email field), placed in 3D. It can sit on the phone's screen, lift off it, fly to the camera, and be **repainted** to show a new state (typed text, a focus ring, a pressed button).

```
const email = new Widget(loginImage, [31, 344, 332, 44], { radius: 22 });   // x, y, w, h in Figma units
email.draw((c) => { c.fillText(typed('pawrent@pawmail.com', lt, 0.25, 0.045), 47, 366); });  // type it
drawPawTap(c, ...email.toFrame(cam, 110, 366), lt - 0.25);                  // the paw that taps it
```

The first test (Sign in, exploded phone) was promising.

Your feedback: *"reduce the video to 30 s … too slow, remember the P(doom) video … start with text that becomes the logo … interactive for all features."*

3.7 Stage 6: the 30-second cut

- **The music cut:** we found a 30.6 s window that needs **no editing**. It starts on a measured downbeat (45.553 s), the drop lands at 1.656 s, and the track's own ending finishes the film.
- **Nine shots of one bar each** (3.311 s), with a change on every beat → `storyboard.launch.json`, generated by a Python script so the times always add up.
- **The cast from the designs:** Bubbly (golden retriever) and Lily (Aussie). So **no new photos** were needed.
- **A shared base** for shots: `scenes/_shot.ts` (camera keyframes, headlines, layers).

3.8 Stage 7: the enhancement pass ("lots of enhancement")

You asked for every enhancement group, plus *"different camera angles and new styles for showing features, not just a mobile screen"*:

Shot	Old idea	New idea	New code
Sign in	exploded phone	**giant keyboard world**	`Keyboard` in `_kit.ts`
Profile	phone lying flat	**polaroid desk** (polaroid, collar tag, medal, trait blocks)	`film-s03.ts`
Feed	one post	**endless scroll wall** + a 3D extruded heart	`film-s04.ts`
Discover	cards fan	**carousel ring**	`film-s05.ts`
Match	loops on a screen	**face to face stickers** + notification banner	`sticker`, `banner` in `_kit.ts`
Chat	side of a phone	**conversation in the air**	`film-s07.ts`

Plus, for all shots:

- **Worlds per act**: gradient + floating paw/loop decor + a shadow floor → `_world.ts`.
- **Slabs**: widgets got real thickness, a sheen and shadows (`Widget.slab()`).
- **Beat punch**: a small zoom on every beat, via the engine's post `zoom`.
- **MSAA**: the 3D stage renders into a 4× multisampled target, for smooth edges.
- **Sound effects**: synthesised in code and placed on every action → `scripts/sfx.ts`.

3.9 Stage 8: making rendering work on your PC

The best design is useless if it can't render. The problems we hit (details in chapter 6):

Problem	Cause	Fix
Rendering ~1 s per frame	The script forced `--use-angle=metal` (a **Mac** flag), so Chrome fell back to **SwiftShader** (software, CPU)	Choose the backend per OS: `d3d11` on Windows. **~6× faster**

Problem	Cause	Fix
"Execution context was destroyed"	The render used a dev server with **live reload**; the page reloaded mid-render	Always use the render script's private server (`--url http://localhost:5999`)
"Target crashed", "fence wait failed"	**GPU memory**: 9 heavy scenes loaded in one page, each with big textures, shadow maps and its own MSAA target, on an integrated GPU	Share one MSAA target, smaller shadow maps, and **render one shot at a time** (`--only S0N`), then join
Doubled text in fast moves	Only 2 motion-blur sub-frames	Adaptive sampling (`--samples auto --max-samples 36`)
Audio peaking at 0 dB	The limiter's auto-level pushed the level back up	`alimiter=...:level=0`
31 leftover servers	On Windows, killing `bunx` doesn't kill its child `node`	Kill the whole process tree (`taskkill /T`)

Lesson: a big part of real engineering is making things work **on the actual machine**, then writing the fix down so nobody hits it again.

3.10 The final pipeline

```
bun scripts/prep-launch.ts                    # music cut + beat grid
for each shot S01..S09:
  bun scripts/render.ts video --only S0N --from <start> --to <end> \
      --samples auto --max-samples 36 --shutter 0.3 --noaudio --out out/parts4/s0N.mp4
ffmpeg -f concat ... → video_only.mp4          # join the 9 shots (1835 frames)
bun scripts/sfx.ts --mux video_only.mp4 MeetMyPets_30s_v3.mp4   # music + effects
```

CHAPTER 4

4. Music and timing

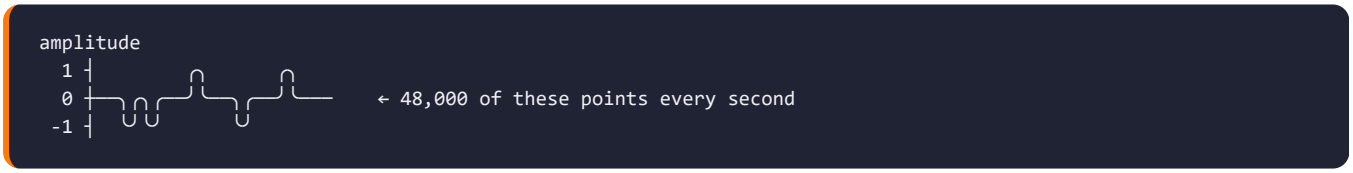
A film feels "professional" largely because things happen **on the music**. This chapter explains how we found *where* the beats are, and how the code uses that.

4.1 Music words, simply

Word	Meaning	In our track
Beat	the pulse you tap your foot to	every **0.828 s**
BPM	beats per minute	60 / 0.828 ≈ **72.5** (a "half-time" feel; double it and it's ~145)
Bar	a group of beats (here 4)	**3.311 s**
Downbeat	the first beat of a bar: the strongest	where our cuts happen
Drop	the moment the full band slams back in after a quiet part	**1.656 s** into our 30 s cut
Breakdown	a quiet section before the drop	the first 1.6 s of the cut

4.2 Sound as numbers

A digital audio file is a long list of numbers (**samples**), 48,000 per second (48 kHz). Each number is how far the speaker cone is pushed at that instant.



Useful measurements:

- **RMS loudness**: take a short window (10 ms), square the samples, average, square-root. That's the "energy" of the window.
- **Decibels (dB)**: a log scale for loudness. Every −6 dB is half the amplitude. 0 dB is the maximum; −40 dB is quiet.
- **Frequency bands**: low (bass, kick drum < 150 Hz), mid (voices, synths), high (hi-hats > 6 kHz). We split them with **filters** (ffmpeg's `lowpass` / `highpass`).

4.3 How we measured the beat grid (no AI needed)

Script: `docs/launch-film/tools/measure_audio.py` (pure Python + ffmpeg, no libraries).

1. **Decode** the MP3 to raw samples with ffmpeg (mono, 12 kHz is plenty).
2. **Envelopes:** RMS loudness every 10 ms, for the full mix and for the low/mid/high bands.

3. **Onsets** (when something *starts*): where the loudness jumps up. That's the positive difference between neighbouring windows ("spectral flux", simplified).
4. **Tempo:** an **autocorrelation** of the onset curve. Slide the curve against itself; it matches best when shifted by one beat. That first guess said ~97 BPM.
5. **The surprise:** 97 was **wrong**. It matched a dotted rhythm (3 sixteenth-notes). The real clue was the **bass**: just before every bar, the bass stops for ~0.33 s and then hits on the downbeat. Listing those bass gaps gave a perfectly regular grid:

```
bass returns at: 2.51  5.82  9.13  12.46  15.76  19.07 …  72.04
differences:        3.31  3.31  3.33   3.30   3.31 …          → bar = 3.311 s
```

1. **Sections** came from loudness per bar: the breakdown (bass at −79 dB) and the drop at 47.21 s, where the full band returns on beat 3 of a bar.

Lesson for an AI engineer: the first algorithm's answer (97 BPM) looked confident and was wrong. We found it by **checking against a second, independent signal** (the bass gaps). Always cross-check model or algorithm outputs.

4.4 How P(doom) did it: the AI audio pipeline

P(doom) had to sync **words**, not just beats. So it used real AI models (files in `analysis/`):

```
song.mp3
   │
   ├─► Demucs (neural source separation) ─► vocals / drums / bass / other stems
   │
   ├─► librosa (signal processing)       ─► beats, downbeats, sections, envelopes   (analyze.py)
   │
   └─► vocal stem
         ├─► CTC forced alignment  (ctc_emissions.py, ctcalign.py, pron.py)
         │     an acoustic model gives, every 20 ms, the probability of each letter;
         │     we KNOW the lyrics, so we find the best path of the known words through those probabilities
         │
         ├─► Whisper (mlx-whisper)  (whisper_run.py)
         │     speech recognition with word timestamps, used as an independent cross-check
         │
         └─► align.py: combine both, refine with vocal features (pitch, onsets) → data/lyrics.json
```

The new words:

- **Source separation (Demucs):** a neural network that splits a mixed song into instruments. Like un-baking a cake into flour, eggs and sugar. Word timing is much easier on the vocals alone.
- **Whisper:** OpenAI's speech-recognition model. It can output a time for each word.
- **CTC (Connectionist Temporal Classification) forced alignment:** "forced" because the text is already known. The model doesn't have to *guess the words*, only *when* each is sung. More accurate than free recognition for singing.
- **Two independent methods + arbitration:** the same cross-checking lesson again.

4.5 From measurements to the film

`scripts/prep-launch.ts` does two things:

1. **Cuts the music**: `ffmpeg -ss 45.553 -t 30.585` plus a 30 ms fade-in (cutting mid-song can click).
2. **Writes the beat grid**: `data/audio.json` gets every beat and downbeat in **film time** (0 = the cut point).

The engine loads that grid, so every frame knows `f.beat` and `f.beatPhase`. Our code uses timing in three ways:

```
// 1. Constants: every shot is one bar; things happen on beats inside it
export const BEAT = 0.82775, BAR = 4 * BEAT;          // _shot.ts
const T = { like: 2.52 };                              // ≈ beat 3 of the bar (3 × 0.828 = 2.48)

// 2. The beat punch: a small zoom that decays after every beat
const zoom = 1 + 0.022 * Math.exp(-f.beatPhase * 9);   // _shot.ts → post.zoom

// 3. Sound effects at the same times as the picture
tap(S(5) + 2.52, 0.4);                                 // sfx.ts: the like tap in shot 5
```

The storyboard is generated by a script that computes every time from the grid (`D(n) = n × BAR`), so shots always start exactly on downbeats and the durations always add up to exactly **30.585 s = 1835 frames**.

4.6 Exercises

1. Open `data/audio.json`. How many beats are in it? Do the downbeats step by 3.311?
2. In `film-s05.ts`, change `T.like` from 2.52 to 1.66 (beat 2). What else must change so the sound still matches? (Hint: `sfx.ts`.)
3. Why did we cut the music at 45.553 s and not at 45.0 s?

Answers

1. 37 beats; yes (0, 3.311, 6.622 …).
2. The like sound cue in `sfx.ts` (`tap(S(5) + 2.52 …)`, the thud at 2.56) and anything timed after the like (the stamp, the rush) in the scene.
3. 45.553 is a measured **downbeat**: starting there keeps the whole grid aligned to t = 0, and the drop lands musically at 1.656 s.

CHAPTER 5

5. The shots, one by one, and the building blocks

5.1 Three coordinate systems (read this first)

Almost every bug in this kind of code is a coordinate mix-up. There are three spaces:

Space	Units	Origin	Used for
Design units	Figma points: the phone frame is **393 × 852**	top-left of the Figma frame	cutting widgets (`x, y, w, h`), painting on them
World units	three.js metres-ish; the phone is 1.5 tall	centre of the 3D scene; **y up**, **z towards the camera**	placing widgets, cameras, lights
Frame pixels	**1920 × 1080**	top-left of the video	2D overlay: headlines, paw taps, flashes

The bridges between them:

- `UNIT = SCREEN_W / 393` : world units per design unit (`_ui.ts`).
- `widget.toFrame(cam, u, v)` : a point **on a widget** (design units) → **frame pixels**, through the camera. This is how a 2D paw lands exactly on a 3D button.
- `Export px = design units × 4` for our @4x Figma exports.

5.2 The building blocks ("kits")

`_shot.ts` : the base of every shot

- `makeWorld(act)` : colour, decor and floor for this part of the story.
- `camera(lt, keys)` : the camera glides between **keyframes** `{ t, pos, at, e }` (`e` = the easing into that key: `outExpo` for snappy, `inOutCubic` for smooth, `inOutExpo` for a whip).
- `headline(c, lt, {...})` : the **tag-type** title: words spring in, accent words hang in a swinging collar-tag pill.
- `punch` : how hard the beat punch zooms.
- Renders the 3D stage through a shared **4× MSAA** target.

`_ui.ts` : widgets

```
const w = new Widget(img, [x, y, w, h], { radius, key, scale, at });
```

- `img` : a Figma export **or** a canvas we drew in code (`src()` in `_kit.ts`).
- `key` : a background colour to **key out** (make transparent): text and logos become floating "stickers".
- `w.draw(paint)` / `w.drawKeyed(key, paint)` : repaint the widget. `drawKeyed` only repaints when `key` changes. **Important for speed**: every repaint re-uploads a big texture to the GPU.

- `w.blurOut(c, rect)`: erase text from a photo by redrawing that patch blurred (how "Ruby · Maine Coon" became "Lily · Aussie").
- `w.slab(depth, colour)`: give it thickness: a rounded 3D slab behind it that casts a real shadow, plus a gloss sheen.
- `w.setWorld(pos, rot, scale)`: place it freely in 3D. `w.place(lift, k, heroPose)`: place it **on a phone** and fly it to the camera (`heroPose`).
- `drawPawTap(c, x, y, age)`: the paw that presses at `age = 0`, with a ripple.
- `typed(text, lt, t0, per)`: how much of a text has "been typed" by now.

`_kit.ts`: drawn-in-code pieces

- `sticker(img, faceRect, d, ring)`: a pet's round photo as a thick disc (Bubbly coral, Lily amber).
- `Keyboard`: a keyboard whose keys light up as you type (`kb.draw(char)`, `kb.keyCenter(char)` for the paw).
- `banner(text)`: the "MeetMyPets · now" notification.
- `pawTrail(...)`: paw prints that walk across the frame.
- `FACES`: where Bubbly's and Lily's faces are in the Figma exports.

`_world.ts`: the world per act

A soft gradient backdrop, 14–22 blurred paw/loop shapes floating in depth (they always face the camera, so camera moves show **parallax**), a **shadow-catching floor**, and a key light with soft shadows.

`_mark.ts`: the logo's two loops

`drawLoop(c, 'L' | 'R', x, y, size, fill, dx, dy, rot)` draws one loop of the official logo, so the two loops can fly in separately and lock together. That's the film's signature move.

5.3 The nine shots

Each shot is one bar (3.311 s). Inside, things happen on beats (0, 0.83, 1.66, 2.48 s).

Shot	File	Idea	Key techniques
S01 Hello	`film-s01.ts`	the line types itself; on the **drop** the letters squeeze, two loops fly in and lock into the logo; the coral burst; the wordmark writes; we zoom through the loop	pure Canvas 2D, a global zoom transform around the loop's hole
S02 Sign in	`film-s02.ts`	a **giant keyboard**; the email and password type with keys lighting; the Login press → spinner → red burst	`Keyboard`, widgets cut from *Login Page*, `currentKey(lt)` keeps keys, text and paws in sync
S03 Your pet	`film-s03.ts`	the red shrinks into the location button; a pin drops; a **polaroid desk**: polaroid, collar tag stamped "Bubbly", Female, trait blocks raining, a medal filling	objects lying on the floor plane (`rotation.x = -π/2`), a `bounce()` easing, `drawKeyed` per state
S04 Feed	`film-s04.ts`	an **endless scroll wall** decelerates onto Lily's post; it pops out; a **3D heart** (extruded `THREE.Shape`) bursts; 111 → 112; Follow → Following	cheap thumbnail textures for the wall, a modulo wrap for "endless", rail buttons as child widgets

Shot	File	Idea	Key techniques
S05 Discover	`film-s05.ts`	a **carousel ring** of cards; Max is flicked away with ✕, Lily gets the ♥	cards on a circle (`sin` / `cos`), the ring angle as a function of time
S06 Match	`film-s06.ts`	**face to face**: the stickers shoot their loops, which lock; flash, hearts, the match screen, a notification; Chat Now swells red	2D loops placed at the projected sticker positions, `globalCompositeOperation = 'lighter'` for the glow
S07 Chat	`film-s07.ts`	**conversation in the air**: typing on a keyboard, the bubble arcs to Lily, typing dots, her reply arcs back	arcs = `lerp` + `sin(πk)` for the height
S08 Everything	`film-s08.ts`	one word and one feature per beat on coral; everything swirls into a point	stronger `punch` , per-beat `setWorld`
S09 End	`film-s09.ts`	the loops slam into the woven logo on the final hit; wordmark, tagline, Coming soon; hold	2D only; the logo SVG as an image

Transitions are designed, not default. Each shot's ending hands over to the next: S02's red burst → S03's red shrinking into a button; S06's red swell → S07's red wipe; S08's white flash → S09.

5.4 Tutorial: change something yourself

A. Change a headline (2 minutes)

1. Open `film-s04.ts` and find `title: 'Like what\nyou love.'` .
2. Change it to `'Stop scrolling.\nLook at Lily.'` .
3. Render a still to check:

```
bun scripts/render.ts stills --url http://localhost:5999 --only S04 --t 11.2 --out ../out/wip/try
```

4. Open `out/wip/try/f_0011.20.png` .

B. Add a second like on Lily's card (15 minutes)

1. In `film-s05.ts` , add `like2: 2.9` to `T` .
2. In `overlay` , copy the `drawPawTap(... lt - T.like ...)` line and use `T.like2` .
3. Add a matching sound in `sfx.ts` : `tap(S(5) + 2.9, 0.4);` .
4. Render stills at 16.1 and 16.2 s and check the paw.

C. Add a whole new shot (an hour)

1. Copy `film-s09.ts` to `film-s10.ts` . Rename the class, keep `extends Shot` .
2. Add an entry to `storyboard.launch.json` with `id: "S10"` , `start` , `end` and `template` . The music must be long enough, and the times must not overlap another shot (or shorten one).
3. `timeline.ts` automatically picks `scenes/film-s10.ts` for `S10` .
4. Render stills with `--only S10` .

Debugging tips:

- **Nothing shows?** Check `w.group.visible`, the camera's `at` point, and whether the widget is behind the camera.
- **Text in the wrong place?** You probably mixed design units and frame pixels.
- **Slow render?** Look for `draw()` called every frame; use `drawKeyed`.

CHAPTER 6

6. Running, rendering, debugging

6.1 One-time setup (already done on your PC)

- **Bun**, **ffmpeg**, **Google Chrome** installed (Windows: winget).
- `cd app` then `bun install`: installs three.js, Vite, Playwright and so on (the `node_modules` folder).
- Fonts in `app/public/fonts/` (Bricolage Grotesque, Nunito Sans, Love Ya Like A Sister).

6.2 Everyday commands (run inside `app/`)

Goal	Command
Live preview in the browser	`bunx vite --port 5173`, then open `http://localhost:5173/?t=10`
Prepare music + beat grid	`bun scripts/prep-launch.ts`
Check the code for type errors	`bunx tsc --noEmit -p tsconfig.json`
Still frames of one shot	`bun scripts/render.ts stills --url http://localhost:5999 --only S04 --t 10.5,11.2 --out ../out/wip/s04`
Speed test	`bun scripts/render.ts perf --url http://localhost:5999 --only S04 --from 11 --to 11.25`
Render one shot as video	`bun scripts/render.ts video --url http://localhost:5999 --only S04 --from 9.933 --to 13.244 --samples auto --max-samples 36 --shutter 0.3 --noaudio --out ../out/parts4/s04.mp4`
Make the sound effects + final mix	`bun scripts/sfx.ts --mux ../out/parts4/video_only.mp4 ../out/MeetMyPets_30s_v3.mp4`

Why `--url http://localhost:5999`? Nothing runs on port 5999, so the render script starts its **own private server without live reload**. A live-reloading server can refresh the page in the middle of a render and kill it.

Preview keys: space = play/pause, ←/→ = ±1 s, `,` / `.` = ±1 frame, `[` / `]` = previous/next shot. Add `?labels=1` to the address to show the review labels.

6.3 The full render, step by step

1. `bun scripts/prep-launch.ts`
2. Render each shot S01…S09 (times from `storyboard.launch.json`), one command per shot. **One at a time**, because all nine together exhaust the integrated GPU's memory. About 2–5 minutes per shot.
3. Join them:

```
ffmpeg -f concat -safe 0 -i ../out/parts4/list.txt -c copy ../out/parts4/video_only.mp4
```

`list.txt` contains lines like `file 's01.mp4'`.

4. Mix the music and effects: `bun scripts/sfx.ts --mux ...`
5. **Verify** (never skip this):

```
ffprobe -v error -show_entries stream=width,height,r_frame_rate,nb_frames:format=duration -of default=nw=1
../out/MeetMyPets_30s_v3.mp4
```

You should see 1920 × 1080, 60/1, **1835 frames**, 30.585 s. Then check for black frames with `blackdetect` and for clipping with `volumedetect`: max_volume should be below 0 dB.

6.4 War stories: every problem we hit and how we solved it

These are worth more than the code. Each one is a debugging story you can tell in an interview.

1. "The render takes 1 second per frame."

- *Symptom:* even a simple 2D shot took 600–1000 ms per frame.
- *Investigation:* a small test script printed which GPU Chrome was using. With the script's flag `--use-angle=metal` it said **SwiftShader** (software rendering on the CPU). Without the flag it said **Intel UHD Graphics**.
- *Cause:* `metal` is a **macOS-only** graphics backend. On Windows Chrome quietly fell back to the CPU.
- *Fix:* choose by OS in `render.ts`: `metal` on Mac, `d3d11` on Windows. The result was **130–220 ms per frame**.
- *Lesson:* **measure, then explain the measurement**. A silent fallback is the worst kind of bug.

2. "Execution context was destroyed."

- *Cause:* a live-reload dev server reloaded the page during the render.
- *Fix:* always let the render script run its own non-reloading server.

3. "Target crashed" and "readFinish: fence wait failed."

- *Cause:* GPU memory. Nine detailed scenes were loaded at once in each of two browser pages: big 4× UI textures, shadow maps, a multisampled target per shot. An integrated GPU shares system RAM and gave up.
- *Fix:* one shared multisampled target, smaller shadow maps, and **render shot by shot** (`--only S0N`). An occasional "fence wait failed" at start-up goes away on a simple retry.
- *Lesson:* design for the **hardware you have**; split big jobs into independent pieces.

4. "The video won't play."

- *Cause:* the render was killed when the session ended, which left an MP4 with no index (the "moov atom"). `ffprobe` says *moov atom not found*.
- *Fix:* render in shorter, independent pieces that each finish, and always `ffprobe` the result.

5. Doubled text in fast moves.

- *Cause:* `--samples 2` gives 2 exposures of fast objects.
- *Fix:* adaptive motion blur (`--samples auto --max-samples 36`).

6. 31 leftover servers.

- *Cause:* on Windows, `bunx vite` starts `node` as a child; killing `bunx` leaves `node` running.
- *Fix:* `taskkill /T /F /PID` kills the whole process tree.

7. Small but sneaky:

- **Bun on Windows** didn't truncate a file it overwrote, so the old file's tail remained and the JSON broke. Fix: delete the file before writing.
- **PowerShell:** a one-item array of pairs got *flattened*, so a patch silently did nothing. Lesson: after automated edits, **check the change actually happened** (search for the new text).
- **.NET file APIs** ignore PowerShell's current folder, so relative paths pointed at the wrong place. Use absolute paths.
- **Audio clipping:** the limiter's auto-level raised the peak back to 0 dB (`level=0` fixed it).

6.5 A debugging checklist

1. **Reproduce small:** one shot (`--only`), one still (`--t`).
2. **Look at it:** render stills and **view the PNG**; make a contact sheet.
3. **Measure:** the `perf` mode for speed, `ffprobe` for files, `volumedetect` for audio.
4. **Change one thing**, re-run, compare.
5. **Write down the fix** (like this chapter).

CHAPTER 7

7. Turning this into a video generator

7.1 The vision

> **Upload your app screens + a music track + one paragraph about your product → get a polished, on-beat, interactive launch film in 16:9, 9:16 and 1:1.**

Today the film is **hand-coded**: each shot file knows exact rectangles, texts and times. A generator needs those to come from **data** that a program (or an AI) can produce.

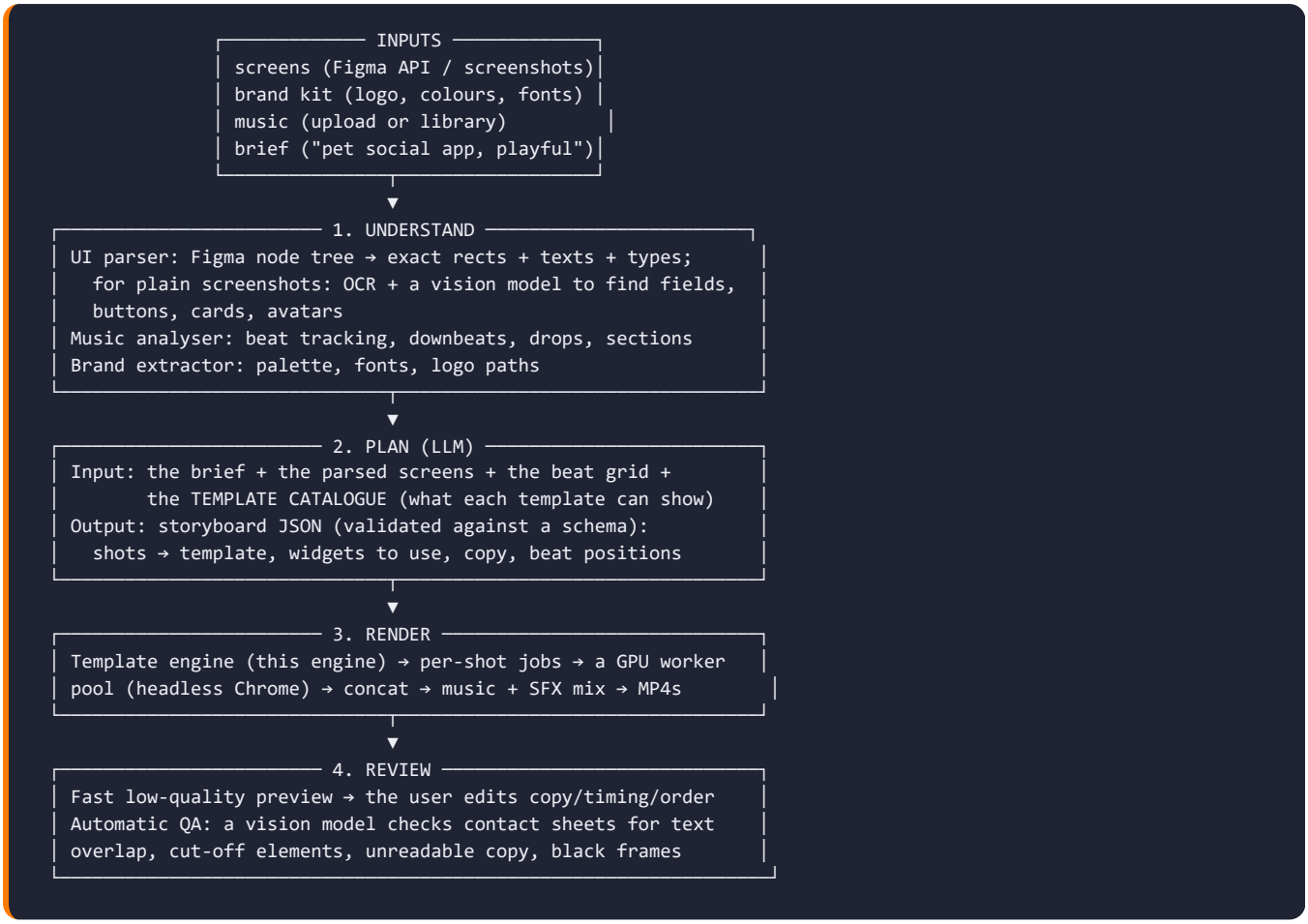
7.2 What we already have (more than you think)

Need	Already built	Where
Render any time `t`, deterministic	✅	`engine.ts`
A storyboard file driving the edit	✅	`storyboard.launch.json` + `timeline.ts`
Beat-locked timing	✅ (measured by hand, applied automatically)	`prep-launch.ts`, `_shot.ts`
Cut UI pieces from screenshots into 3D	✅	`_ui.ts` `Widget`
Reusable visual devices	✅	`_kit.ts`, `_world.ts`, `_mark.ts`
Sound effects from a cue list	✅	`sfx.ts`
Headless rendering to MP4	✅	`render.ts`

7.3 What's missing

1. **Templates instead of one-off shots.** "Keyboard world", "polaroid desk", "scroll wall", "carousel", "face to face", "chat in the air", "montage" and "end card" should each become a **template** that takes parameters (which widget rects, which texts, which photos, which beats).
2. **Automatic understanding of the inputs:** where the buttons and fields are, the brand colours, the beats.
3. **Automatic planning:** which templates, in which order, with which copy.
4. **A UI** to preview and tweak, and **cloud rendering**.

7.4 Target architecture



7.5 The AI parts in detail

A. UI understanding. Best source: the **Figma REST API**. It returns every layer with its exact position, size, text and style, so there's no guessing. For plain screenshots: OCR for the texts, plus a **vision-language model** asked to return JSON like `{type: "text_field", rect: [31, 344, 332, 44], label: "Email"}`. Validate with simple rules (rects inside the frame, no overlaps).

B. Music understanding. Use our measurement method as a baseline, plus a dedicated beat-tracking model (open-source beat/downbeat trackers exist) for robustness. Detect **drops** as big jumps in low-band energy after a quiet stretch. Put the drop on the product's "magic moment" (our match).

C. Planning with an LLM. The key design: the LLM does **not** write code. It writes **data** that is checked against a **schema**:

```
{
  "shots": [
    { "template": "keyboard_world", "bars": 1,
      "widgets": { "email": "login.email_field", "password": "login.password_field", "cta": "login.button" },
      "copy": { "kicker": "Sign in", "title": "Settle in.", "accent": "in." } },
    { "template": "face_to_face", "bars": 1, "on_drop": true,
      "pets": ["user_pet", "match_pet"], "copy": { "title": "The vibe is mutual." } }
  ]
}
```

- Give the model a **catalogue** of templates with what each needs and shows.

- Use **structured output** (JSON-schema mode), then **validate** with a schema library (e.g. zod in TypeScript). On failure, send the errors back and retry.
- Rules in the prompt: *only features present in the screens; ≤ 4 words per title; one feature per bar; the match on the drop.*
- **Evaluate**: build a set of ~20 test briefs and check that the JSON is valid, the features are real (no hallucinations), the copy length is right, and humans rate the results.

D. Copy and voice. An LLM writes headlines in the brand's tone. Optional text-to-speech voiceover and subtitles for other languages.

E. Where generative video fits. AI video models are good for **atmosphere** (b-roll, backgrounds, pet footage) but bad at **exact UI** (they distort text and invent buttons). Keep the product deterministic, and use generative video only for the supporting footage.

7.6 Phased roadmap

Phase	Goal	Main work	Duration (solo)
0. Refactor	shots become templates	move rects/texts/times from `film-sNN.ts` into JSON params; one generic file per template	2–3 weeks
1. CLI generator	`generate --screens ./figma --music song.mp3 --spec plan.json`	automatic beat grid (generalise `measure_audio.py`), per-shot render + concat + SFX in one command	2–3 weeks
2. AI planner	brief → plan.json	LLM + schema + validation + a template catalogue; a test set of briefs	3–4 weeks
3. UI parsing	screens → widget library	Figma API import; screenshot fallback with OCR + vision	3–4 weeks
4. Web app	upload, preview, edit, download	React UI, low-res live preview (the engine already previews in the browser), job queue	6–8 weeks
5. Cloud	many users	GPU workers (headless Chrome with a real GPU), a queue (e.g. Redis-based), storage, billing	ongoing

7.7 Build vs reuse

- **Remotion** (React-based programmatic video, with cloud rendering) is the closest existing tool. Our engine has stronger 3D, motion blur and post effects; Remotion has a big ecosystem and serverless rendering. A generator could keep our engine or port the templates to Remotion. Compare the two before you build infrastructure.
- **Theatre.js** (a keyframe editor for three.js) could give designers a timeline UI over our scenes.

7.8 Risks to plan for

- **Licensing:** music, fonts and app-store badges must be licensed for commercial use.
- **Accuracy:** never show features the app doesn't have (we checked against the real code). In a generator, enforce this with the parsed UI.

- **Render cost and time:** about 40 minutes on an integrated GPU for 30 s at top quality; a cloud GPU is much faster. Offer "draft" (`--samples 1`) and "final" quality.
- **GPU memory:** render shot by shot (lesson from chapter 6).

7.9 Your first three tasks, if you start tomorrow

1. Move **S05 (carousel)** into a generic `template-carousel.ts` that reads its cards, texts and beats from JSON. Prove it by making a version with different pets.
2. Turn `measure_audio.py` into a function that outputs `timing_basis` for **any** track.
3. Write a script that renders all shots of a storyboard, joins them and mixes the audio in **one command** (what we did by hand in chapter 6).

CHAPTER 8

8. Your career guide: explaining this project and growing from it

This chapter turns the project into **career material**: pitches, interview answers, honest skill notes and a learning plan.

8.1 What this project actually proves about you

Skill	Evidence in this project
Reading and extending a large unfamiliar codebase	built on the P(doom) engine instead of starting from zero
Graphics and 3D (three.js, WebGL, shaders basics)	widgets as 3D planes, shadows, MSAA, post effects
Signal processing and audio AI	beat grid measurement, Demucs / Whisper / CTC in P(doom)
Automation and pipelines	headless Chrome → ffmpeg, scripted storyboard, synthesised SFX
Debugging on real hardware	SwiftShader fallback, GPU memory crashes, process trees
Working from feedback	five big direction changes, each handled without a rewrite
Product thinking	showing only real features, story from sign-in to match

8.2 The pitches

30 seconds (elevator):

> "I built a 30-second launch film for a pet social app, entirely in code. It's a deterministic rendering engine in TypeScript and three.js: every frame is a function of time, so a headless browser can render it frame by frame into ffmpeg. The app's real Figma screens are cut into 3D widgets that a paw 'taps', and everything is locked to a beat grid I measured from the music. The interesting part was engineering: I found a 6× speed-up from a wrong GPU backend flag and fixed GPU memory crashes by restructuring the render."

2 minutes (interview "tell me about a project"):

1. **Context:** a startup needed a launch video; I had a music-video engine (P(doom)) that synced visuals to lyrics using AI audio models.
2. **Goal:** a 30 s, YouTube 16:9 film where every feature is shown interactively, on the beat.
3. **What I built:** a widget kit (Figma crops → 3D objects with state), shot templates with camera keyframes, per-act 3D worlds, a measured beat grid, synthesised sound effects, and a render pipeline.
4. **Hard problems:** the beat detector said 97 BPM and was wrong; I cross-checked with bass gaps and found the true bar of 3.311 s. Rendering was 1 s/frame because a Mac-only flag forced CPU rendering on Windows; fixing it gave 6×. GPU crashes → shared buffers and per-shot rendering.

5. **Result:** a 1835-frame 60 fps film, and a clear roadmap to turn it into an AI video generator (LLM writes a validated storyboard JSON; the engine renders it).

8.3 STAR stories (Situation, Task, Action, Result)

STAR is the standard way to answer "tell me about a time when…".

1. Debugging a performance problem

- *S:* rendering took about 1 second per frame; a 30 s film would take hours.
- *T:* find out why and make it fast enough to iterate.
- *A:* I didn't guess. I wrote a tiny script that printed which GPU Chrome actually used. It said SwiftShader (CPU). The render script forced a macOS-only graphics flag; I made the flag OS-specific.
- *R:* 130–220 ms per frame, about 6× faster. Lesson: measure first.

2. A confident but wrong algorithm

- *S:* automatic tempo detection said 97 BPM.
- *T:* get a beat grid I could trust for every cut.
- *A:* I noticed edits didn't feel on the music, so I looked at an independent signal: the bass drops out before every bar. The gaps were perfectly regular: 3.311 s.
- *R:* every cut lands on a downbeat. Lesson: cross-check model outputs with an independent signal. Directly relevant to AI work (evaluation, hallucination checks).

3. Working with limited hardware

- *S:* renders crashed with "Target crashed" on an integrated GPU.
- *T:* make the full film render reliably.
- *A:* found the cause (GPU memory: nine scenes with large textures, shadow maps and MSAA buffers in each browser page), shared one multisampled buffer, shrank shadow maps, rendered per shot and joined with ffmpeg.
- *R:* reliable renders; failures became local and cheap to retry.

4. Handling changing requirements

- *S:* the format changed from 76 s vertical to 16:9 to 30 s; the style changed from "phone on screen" to "interactive widgets in 3D worlds".
- *T:* deliver without restarting each time.
- *A:* kept one source of truth (the storyboard JSON, generated by a script from the beat grid) and reusable kits, so changes were data edits, not rewrites.
- *R:* each direction change took hours, not days.

8.4 Interview questions with answers

Q: What does "deterministic rendering" mean and why does it matter? A: The frame at time `t` depends only on `t` (and fixed data), not on how fast the computer is or what came before. So preview and export match exactly, any frame can be rendered alone, and frames can be split across workers.

Q: Why render in a browser instead of a video tool? A: The browser gives a mature GPU API (WebGL), text rendering, fonts and a live preview for free. Playwright makes it scriptable. The trade-off is overhead in moving pixels out (we use a WebSocket to ffmpeg).

Q: How does motion blur work here? A: For each output frame we render several sub-frames inside the "shutter" time and average them. Adaptive mode only adds samples to 32-pixel tiles that actually move, to save time.

Q: How would you add AI to this? A: An LLM planner that outputs a storyboard JSON, validated against a schema, with a catalogue of templates; a vision model to parse screens into widgets; a beat tracker for any music; a vision model for automatic QA on contact sheets. The renderer stays deterministic so the product UI is pixel-exact.

Q: What is CTC forced alignment? A: An acoustic model outputs letter probabilities every ~20 ms. Because we already know the lyrics, we search for the best time path of those known letters through the probabilities (dynamic programming). It gives word timings more reliably than free recognition.

Q: How do you know your AI output is correct? A: Use an independent check: a second method (Whisper vs CTC), a different signal (bass gaps vs onset tempo), schema validation, and a small labelled test set. Measure, don't trust.

Q: What would you do differently? A: Make templates data-driven from the start; write automated visual tests (render stills and compare to references); check the GPU backend on day one.

Q: How did you handle the GPU memory issue? A: Profiled the cause (textures, shadow maps, MSAA targets per scene × 2 pages), shared resources, reduced sizes, and changed the job structure (per-shot renders). Also added retries for a rare startup failure.

8.5 Be honest about what you did

Interviewers dig. Say clearly which parts you designed, which you reused (the P(doom) engine and its AI audio pipeline) and which you built with an AI coding assistant. That is normal now; what matters is that **you can explain every part**: this guide exists for that. Practise by explaining chapter 2 out loud without notes.

8.6 Gaps to fill (honest skills map)

Area	You now know	Learn next
Python and ML basics	reading pipelines	NumPy, pandas, PyTorch, training a small model
Deep learning	what Demucs/Whisper do	how transformers work; fine-tuning; embeddings
LLM engineering	the planner idea	prompt design, structured output, tool use, RAG, evaluations
Computer vision	image crops, compositing	detection/segmentation models, OCR, vision-language models
Audio	beats, RMS, filters	STFT, mel spectrograms, librosa
Software engineering	TypeScript, scripts	testing, Git workflow, code review, CI
Deployment	local renders	Docker, a cloud GPU, queues, APIs (FastAPI)

8.7 A 12-month plan for a fresher AI engineer

Months	Focus	Concrete output
1–2	Python, NumPy, Git, Linux basics; finish one ML course	a notebook re-implementing the beat-grid measurement in Python
3–4	Deep learning foundations (PyTorch): MLPs, CNNs, transformers	train a small image classifier; write what you learned
5–6	LLM engineering: APIs, structured output, tool use, evaluations	**the AI storyboard planner** from chapter 7 with a test set
7–8	Vision: OCR + a vision-language model on app screenshots	screenshot → widget JSON parser, measured accuracy
9–10	Deployment: FastAPI, Docker, a job queue, a cloud GPU	"upload screens + music → video" running online
11–12	Polish, write-ups, open source, applications	a portfolio site, 2 blog posts, demo videos

8.8 Portfolio and job-search tips

- **Show the video first**, then the architecture diagram, then one hard problem you solved.
- Write a **README** with: what it does, a GIF, how to run it, an architecture diagram, results with numbers (6× faster, 1835 frames).
- Write **one blog post per war story**. Short, specific posts get read.
- On your CV, use numbers: *"Cut render time 6× by diagnosing a GPU backend fallback."*
- Keep learning in public: GitHub commits, short posts, small demos.

CHAPTER 9

9. Glossary

Every technical word in this guide, in plain language.

Term	Plain meaning
Animatic	a rough draft of a film, used to judge timing and story before the polish
Autocorrelation	sliding a signal against itself to find how often it repeats (used for tempo)
Bar	a group of beats (here 4 beats = 3.311 s)
Beat / BPM	the pulse of the music / how many pulses per minute
Beat punch	a tiny zoom on every beat that makes the picture "pulse" with the music
Bloom	a glow around bright parts of the image
Bun	a fast JavaScript/TypeScript runtime; runs our scripts
Canvas 2D	the browser's simple drawing API: rectangles, text, images
CI (Continuous Integration)	servers that test code automatically on every change
Chromatic aberration (CA)	colours separating slightly at the edges, like a real lens
Contact sheet	many still frames in one image, to review a shot at a glance
CTC forced alignment	finding *when* known words are sung, using letter probabilities from a model
d3d11 / Metal / ANGLE	graphics backends: Direct3D 11 (Windows), Metal (Mac); ANGLE translates WebGL to them
Demucs	a neural network that splits a song into vocals, drums, bass and other
Design units	Figma's coordinate units; our phone frame is 393 × 852
Deterministic	the same input always gives the same output (frame = function of time)
Downbeat	the first, strongest beat of a bar
Drop	the moment the full music returns after a quiet section
Easing	how motion speeds up or slows down (e.g. `outExpo` = fast start, soft stop)
Extrude	turn a flat 2D shape into a 3D object by giving it depth
ffmpeg / ffprobe	the command-line tool for video/audio processing / for inspecting files
Figma	the design tool where the app's screens were made
Frame / fps	one still image of the video / frames per second (we use 60)
Gloss / sheen	a soft light highlight that makes a surface look shiny
Headless browser	a browser running without a window, controlled by a program
HDR / linear colour	rendering with values above "white" and in physically correct maths before converting for the screen

Term	Plain meaning
HMR (Hot Module Reload)	a dev server feature that updates the page when code changes
Keyframe	a stored value (e.g. camera position) at a time; the code fills in between
Keying (key out)	making one colour transparent, like a green screen
LLM	Large Language Model, e.g. Claude; reads and writes text
libx264 / H.264	the common video compression format and its encoder
librosa	a Python library for audio analysis
Mesh / material / texture	a 3D shape / how its surface reacts to light / an image painted on it
moov atom	the index inside an MP4; without it the video won't play
Motion blur	the smear of fast motion; we create it by averaging sub-frames
MSAA	multisample anti-aliasing: smooth edges by sampling each pixel several times
Onset	the moment a sound starts
Parallax	near things move more than far things when the camera moves
Playwright	a library to control browsers from code
Post-processing	effects applied to the finished image (bloom, grain, vignette, zoom)
Premultiplied alpha	storing colour already multiplied by transparency; mixing the two conventions causes dark edges
Render target (RT)	an off-screen image the GPU draws into
RMS	a measure of loudness over a short window
Schema / validation	a formal description of valid data / checking data against it
Shadow map	an image from the light's point of view used to compute shadows
Shot	one continuous piece of the film between cuts (we have 9)
Shutter	the fraction of a frame's time used for motion blur
SFX	sound effects
sRGB	the standard colour space for screens
Storyboard	the plan of shots with timing; here a JSON file that drives the film
Structured output	asking an LLM to answer in a fixed JSON format
SwiftShader	Chrome's software (CPU) renderer: works everywhere but slow
three.js	a JavaScript 3D library on top of WebGL
TypeScript	JavaScript with types, which catch mistakes before running
Vignette	darkening at the image corners to focus attention
Vision-language model	an AI model that understands images and text together
Vite	a fast dev server and bundler for web projects

Term	Plain meaning
WebGL	the browser's API for using the GPU
WebSocket	a two-way connection between a page and a program; carries our frames to ffmpeg
Whisper	a speech-recognition model that can give word timestamps
Widget	in this project: a crop of a Figma screen turned into a 3D object with states
World units	the 3D scene's units (y up, z towards the camera)